

IoT Door Sensor Project

Submission Date: 4/23/24

Group Members: Sean Hinton (sahinto2), Doc Bojo (dmbojo), Matt Kosmala (mtkosmal), Nick Colannino (nmcolann)

Course: CSC 453 Introduction to Internet of Things

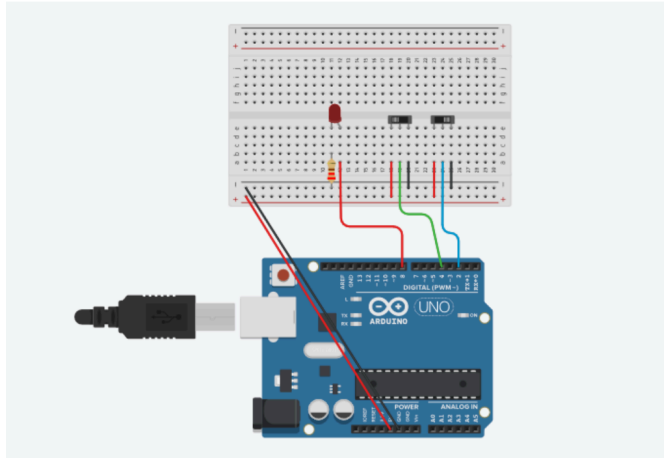
Contributions

Member	Contribution %	Items
Sean Hinton (sahinto2)	25%	• Flask website setup for main.py, index.html, and index.js. • Report Discussions section.
Doc Bojo (dmbojo)	25%	• Arduino code design, setup, and debugging. • Report Implementation - Software section.
Matt Kosmala (mtkosmal)	25%	• Hardware design and setup. • Report Implementation - Hardware section.
Nick Colannino (nmcolann)	25%	• Python CoAP server and Python Serial Listener. • Report Results section.

Designs

This section will discuss the Tinkercad design for the Arduino microcontroller and the system design.

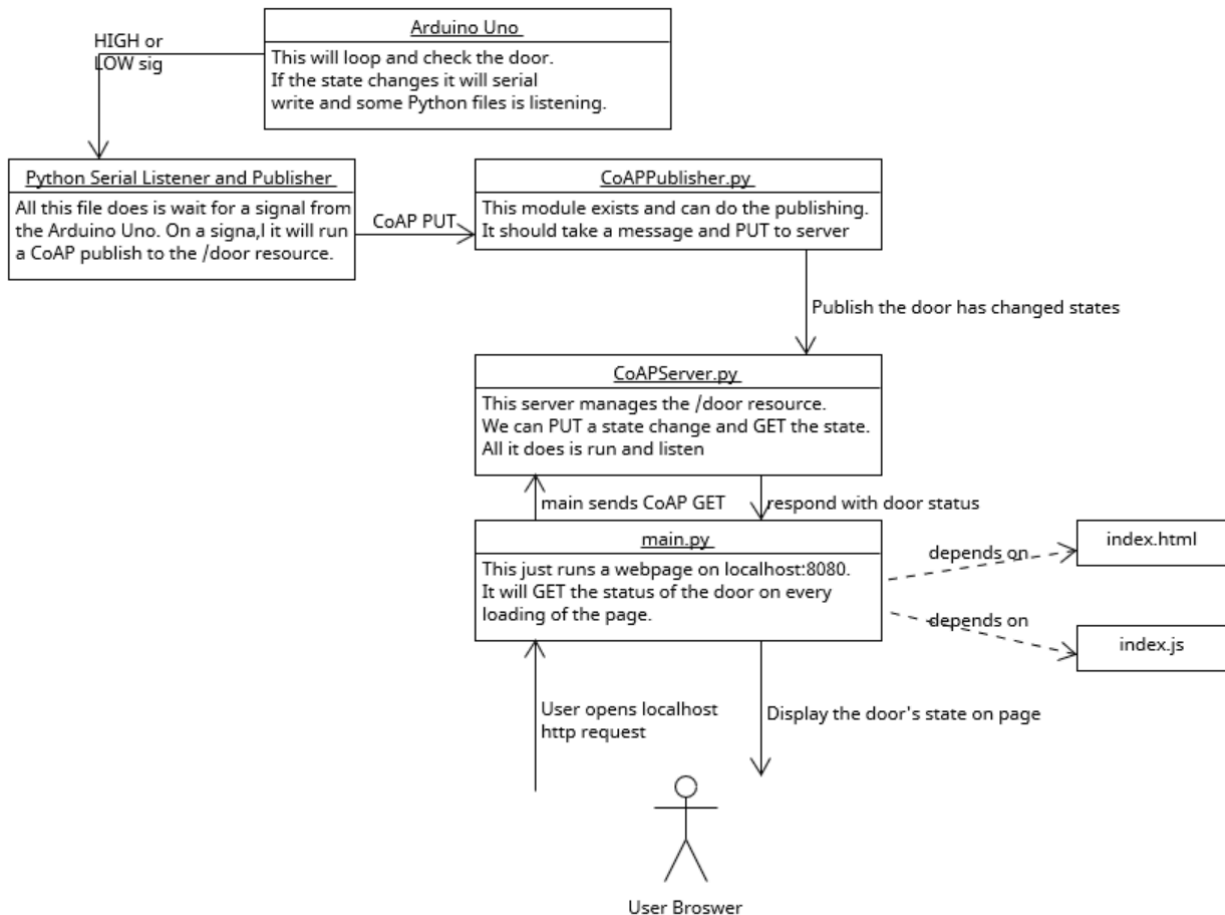
Microcontroller Design



In the microcontroller design, the Arduino has a switch connected to an input and ground pin. The Arduino routes a power supply to the switch and checks if the switch is on or off with the green wire. This ultimately lets us know whether the door has been opened or closed. If the switch is on that tells the Arduino to send some power to the LED and light it up. When the switch is off the light is turned off. There is a second switch connected by the blue wire, and this switch functions as an arming switch. The Arduino will not send any information to the LED if the switch connected to the blue wire is off. This served as a parallel design to our implementation of the reed switch door sensor and web page.

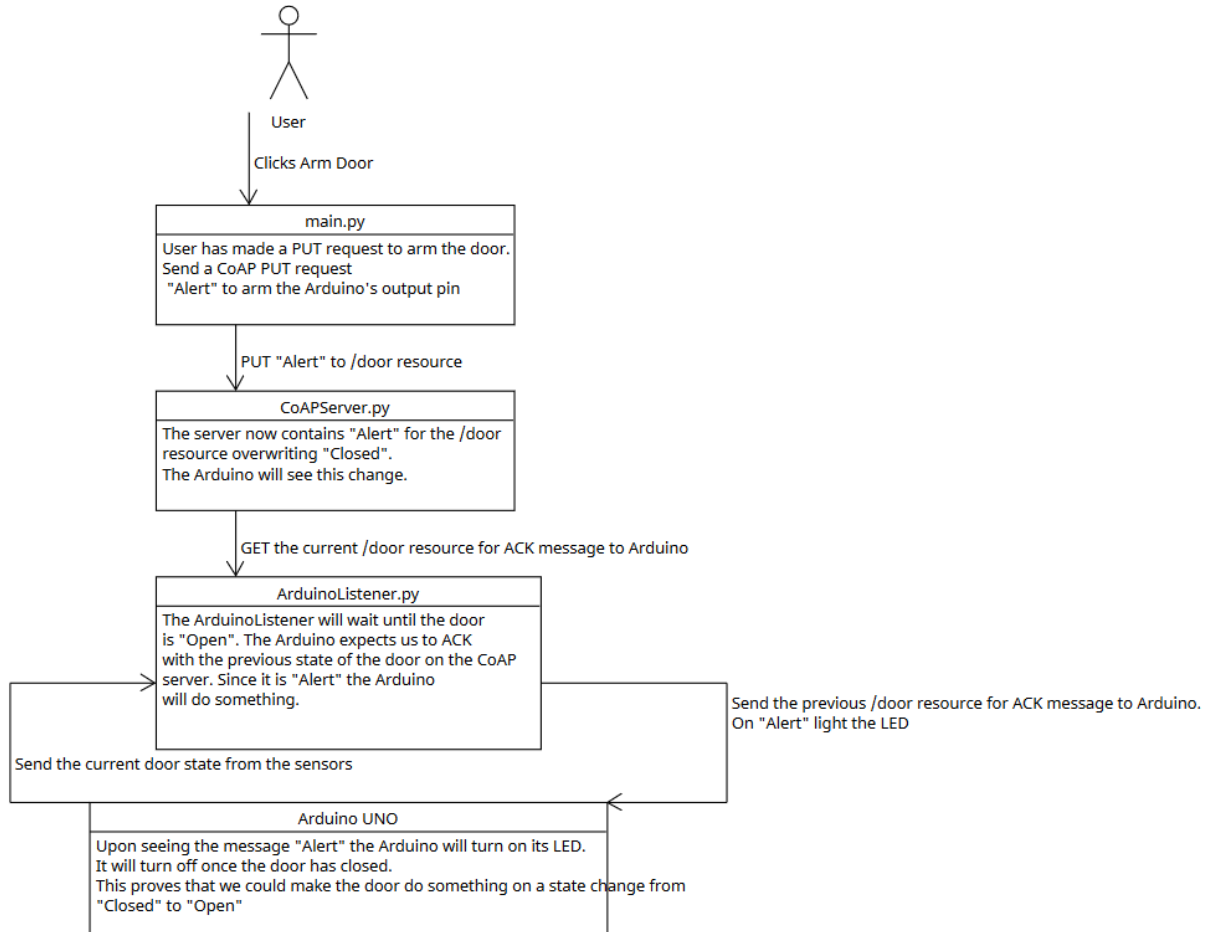
The reed switch circuit can be represented by the switch connected with the green wire. It is responsible for sending signals to some output source. The blue switch functions as an arming switch that will alert the Arduino if something has changed.

System Design



The system design works by using an http proxy to make CoAP requests. The webpage created by main.py will send http GET or PUT requests that make CoAP GET or PUT requests. The CoAPServer.py and ArduinoListener.py work in the background to communicate with the Arduino UNO and make sure the /door CoAP resource is up to date. Overall, the system uses http to CoAP proxies to allow a web page to see the state of the door and access CoAP resources for a user through a web browser. This is much easier for users compared to using a command line interface or forcing them to create a CoAP subscribing program.

ACK Design



To make the alert to the Arduino work we needed a method of updating the Arduino. We decided to implement a call and response to acknowledge the reception of an update from the door sensor. The acknowledgement would be the previous state of the door. The states of the door now included "Alert" which can be activated in the "Closed" state by an http request from the user. In "Alert", the next time the door is moved to "Open" an LED turns on until the door is closed. Another way of implementing this could be with another resource called /door/alarm and simply say "On" or "Off". If the alarm is on and the door is opened, activate the LED. This is a proof of concept that this system can become armed to send power to an alarm or to a motor that automatically closes the door.

Implementation - Hardware

To implement the Arduino we purchased the following components. The screw terminal is optional and makes connecting the wires easier. It also prevents the wires from falling out of the pins.

- 1 Arduino UNO
- 1 Door Sensor (Reed Switch and Magnet)
- 1 USB A male end to USB B male end cable
- 1 Screw Terminal for Arduino UNO (Optional)

The Arduino Uno serves as a foundation of the microcontroller design and implementation. Its digital and analog I/O pins interface with the other external hardware. For this implementation, the door sensor, which consists of a read switch and a magnet, is connected to one of the digital pins in the Arduino. So when the door is closed, this means the reed switch is in proximity to the magnet which in turn creates a closed circuit, and once the door opens, the circuit is broken, which is detected by the Arduino and a message is sent to the CoAP server from the code we wrote on the Arduino.

The USB-A male to USB-B male cable provides both power and data transfer capabilities for the Arduino Uno. It connects to the USB-B port on the Arduino and to a USB-A port on a compatible USB charger or a computer.

As previously mentioned, the screwboard is used to hold wires in place. This part is not necessary, but it is recommended since it can be difficult to connect the wires otherwise. The screw terminal attaches to the Arduino's pins and allows the wires to be screwed in place without the need for soldering, which is useful when this device would hang from a wall or have pins removed and reattached periodically.

In the microcontroller design, the Arduino has a reed switch connected to an input and ground pin. The Arduino routes a power supply to the reed switch and checks if the circuit is closed or open, which ultimately lets us know whether the door has been opened or closed. The reed switch circuit can be closed by placing a magnet next to it or opened by removing the magnet and opening the switch.

Implementation - Software

Arduino

The Arduino code for the door monitor itself is fairly simple. It listens for HIGH and LOW signals. First, a few global variables are set for use in the rest of the implementation:

```
Unset
// Pin for the door sensor
const int reedSwitchPin = 2; // Pin connected to the reed switch

// State
```

```

int reedSwitchState = 1;

// Previous state
int reedSwitchPrevious = 0;

// State to ensure we are acknowledging the door state update
boolean ackSend = false;

```

We set `reedSwitchPin` to correlate with our hardware setup. Then, we keep track of both the current and previous states of our reed switch. Finally, we have a boolean flag that is used to make sure we acknowledge any updates to the door state.

The `setup()` function is shown below:

```

Unset
void setup() {
  Serial.begin(9600); // Initialize serial communication
  pinMode(reedSwitchPin, INPUT_PULLUP); // Set reed switch pin as input with
  internal pull-up resistor
  pinMode(LED_BUILTIN, OUTPUT); // Turn the LED on if you want to see things
  easier

  digitalWrite(LED_BUILTIN, LOW);

  //Do something here to check what state the door is in initially
  reedSwitchState = digitalRead(reedSwitchPin);

  //Send an initial message to clear out the port
  Serial.println("<Now Sending Content>");

  if(reedSwitchState == LOW){
    Serial.println("<Closed>");
  }
  else{
    Serial.println("<Open>");
  }
}

```

`setup()` first initializes serial communication, setting the data rate to 9600 bps. It then designates the reed switch pin as input with the internal pull-up resistor and turns the LED on for visual aid.

Next, the `digitalWrite(LED_BUILTIN, LOW)` command turns off the built-in LED on the Arduino board. After setting up the pins and LED, we proceed to read the initial state of the reed switch with `reedSwitchState = digitalRead(reedSwitchPin)`. This line checks whether the door is initially open or closed based on the state of the reed switch. The subsequent print command sends a message indicating that the Arduino is about to transmit information.

Finally, the ending if-else statement evaluates the `reedSwitchState` variable:

- If `reedSwitchState` is LOW, it means the door is closed, and the corresponding message is sent.
- Else, if `reedSwitchState` is HIGH, it means the door is open, and the corresponding message is sent.

The `loop()` function is shown below:

```
Unset
void loop() {
    reedSwitchPrevious = reedSwitchState;
    reedSwitchState = digitalRead(reedSwitchPin); // Read the state of the reed
switch pin

    //OPEN STATE
    if (reedSwitchState == LOW && reedSwitchPrevious == HIGH) {
        //Closed door

        Serial.println("<Closed>"); // Print message if reed switch is closed (ON)
        digitalWrite(LED_BUILTIN, LOW);
        ackSend= true;

    }
    //CLOSED STATE
    else if(reedSwitchState == HIGH && reedSwitchPrevious == LOW) {
        Serial.println("<Open>"); // Print message if reed switch is open (OFF)
        //digitalWrite(LED_BUILTIN, HIGH);
        ackSend= true;
    }

    //Look and see that the ArduinoListener.py acked our message
    if(ackSend && Serial.available() > 0){

        //Read from the serial port character by character until the new line char
appears
        String cur = Serial.readString();

        cur.trim();

        // Light the LED if needed
```



```

    if(cur.equals("<Alert>")){
        digitalWrite(LED_BUILTIN, HIGH);
    }

    // we have read all of the string so make ackSend false and end this loop
    ackSend = false;

}

delay(500); // Delay for stability. If we read too fast it gets hard to
consistently read messages
}

```

`loop()` first reads and updates state by storing the current state of the reed switch into `reedSwitchPrevious`, which is used to track changes in the state of the reed switch. It then reads the current state of the reed switch pin and updates the reed switch state accordingly. The tracking of door state works like an atomic lock where we overwrite the lock's state and check the previous value for any changes.

When in a LOW state, the reed switch is sitting next to the magnet and the door is closed. We start with the door set as either open or closed initially, so that you have to close or open the circuit in order to see the results.

Next, the function checks if the state of the reed switch has changed from the previous loop iteration, as documented below:

Closed State (`reedSwitchState == LOW && reedSwitchPrevious == HIGH`):

- `reedSwitchState == LOW` means that the door has been closed.
- `reedSwitchPrevious == HIGH` means that the door was open in the previous iteration.
- The Arduino prints a message accordingly.
- It will also turn off the LED and set `ackSend` to true to indicate that a message has been sent.

Open State (`reedSwitchState == HIGH && reedSwitchPrevious == LOW`):

- `reedSwitchState == HIGH` means that the door has been opened.
- `reedSwitchPrevious == LOW` means that the door was closed in the previous iteration.

- The Arduino prints a message accordingly.
- `ackSend` is set to true to indicate that a message has been sent.

After updating the state and printing the appropriate messages, a check is made to see if there is any data available on the serial port. The expected data contains the previous CoAP resource value for `/door`. If there is data available, it reads the incoming messages, character by character, until it reaches the newline character. It then trims any leading or trailing whitespace from the received message.

If the received message is "`<Alert>`", it lights up the LED. Finally, `ackSend` is set to false to indicate that the acknowledgement is complete, and it adds a delay of 500 ms to stabilize the loop and prevent it from reading messages too quickly.

Python

For the Python components of the project, each Python file runs its own process in the background and is required for the system. The components are the Flask application `main.py`, `CoAPserver.py` to manage the state of the `/door` resource, and `ArduinoListener.py` to wait for serial communication from the Arduino UNO. We will show you some important code snapshots for these Python files since the files can get quite large.

The `ArduinoListener.py` serial communication logic.

Python

```
message = serialPort.readline().decode().strip()

content = message.split('<')[1].split('>')[0]

print(content)

# Get the coap resource's contents first before updating
prev = publish_content(content)

# Now ACK the message from the Arduino or send some data

# We will respond with the previous resource from the /door endpoint

# If it is an <Alert> we will have to do something on the Arduino device
```

```
prev = '<' + prev.payload + '>'

serialPort.write(prev.encode('utf-8'))
```

This snapshot shows how the logic for communication between the Arduino and the CoAP server will be managed. A Python file is waiting for communication from an Arduino device. When there is a message, we update the /door resource and send back the previous state of the door. This subroutine will run continuously with a call and response of the door's state until the process is terminated.

The CoAPServer.py /door endpoints.

```
Python

def render_GET(self, request):

    return self

    # Get the message from the request

def render_PUT(self, request):

    self.payload = request.payload
    # Return the payload of the message to the subscriber
```

This section shows how the endpoints for the /door resource will work. On GET requests, it simply returns the whole object for the /door resource. On PUT requests, it updates the payload or text for the /door resource. The server is simple and only manages the /door resource.

The main.py http endpoints.

```
Python
```

```

@app.route("/door/alarm", methods=['PUT'])

def alarm_door():

    content = request.get_json(silent=True).get("state") # GEt the state of the
    door and change the coap request

    print(content)

    alert_data(host,port,content)


    return "Message sent", 200


# Just get the status of the door and jsonify it

@app.route("/door", methods=['GET'])

def get_door():


    state = subscribe_data(host, port)


    # If the status is none type just say door is unknown

```

```

if state is None:

    json_data = {'status': 'idk'}

    return jsonify(json_data), 200


    json_data = {'status': state}


    # We got the door data and return 200

    return jsonify(json_data), 200

```

These are some of the API endpoints for http requests on the web page. The http request acts as a middle man for managing the /door resource. On PUT requests to /door/alarm, the door's state changes to "Alert", and on GET requests to /door, the web page returns the current state of the door. It is quite simple and relies on the CoAP server in the background to serve content to clients, and it uses http request to easily access information.

Results

Our IoT Door Sensor Project revolves around the functionality of a few different technologies such as HTTP, CoAP, Python, and Arduino hardware. The primary goal that we had for this project was to develop a web-based interface that interacts with a physical IoT device to notify a user when their door is open. This goal was achieved by implementing a CoAP server that runs continuously in the background. Here it stores status information about the door which is then retrieved and displayed by the front-end of our system with a flask application.

A quick overview of how the process above works can be shown through examining a few different sections of our project. First, the Arduino listener waits for signals from the Arduino and then creates a PUT request to the CoAP server to update it with the new door status. The Web Server is then able to make a GET request to the CoAP server to notify users within the front-end of our application if their door is open. This front-end application can be accessed at 'localhost:8080' and it displays the current status of the door. Refreshing the interface should show real-time updates in the doors status which are detected by the Arduino sensor.

The two main results that we tested for our project were the reliability of the system and how fast it can interact with the front-end. Throughout testing, the system proved to be stable and reliable, with no crashes or missed updates from the sensors. The system was also able to interact with the web-page immediately upon refreshing the page. This shows effective communication between the hardware and web application. The website also refreshes every 30 seconds and when the door LED is armed or disarmed. The testing of the door's LED was also successful and only lit up when in the "Alert" stage and turned back off when closing the door.

Discussion

The most interesting part of this project was the ability to connect different technologies like http requests and CoAP requests. From a web browser clients can access a CoAP resource just like a url if the correct endpoints are implemented. This gives us the ability to use a more well known technology like http requests and web browsers as a proxy to using a CoAP client and working with command lines. This gives us communication between Arduino controller, a listening process on a server, the CoAP server, the web server, and the browser.

Another extension of this project could allow the user to communicate with the Arduino. We were able to complete this ability to output some signal by arming the door in a closed state. We do this by asking the user if they want to arm the door when in the closed state. If they click a button the CoAP resource changes from "Closed" to "Alert". We also need the Arduino to know when the resource has changed, and this is done by receiving an ACK message when serial writing a state change. The received ACK can be anything, but if it contains ACK it sends some power to an LED or output pin. Currently, it is a proof of concept, but it could be used to route power to a motor to close the door or sound an alarm. The system could also be modified to use another CoAP resource /door/alarm to keep a continuous alarm.

References

Door Sensor Amazon link

https://www.amazon.com/dp/B0B3D7BM4K?&linkCode=sl1&tag=zlufy-20&linkId=ebc9d95cfe52506a8cf7ff5ed3ea73d3&language=en_US&ref_=as_li_ss_tl